

Client-сервер — язык и база данных

Дата: 2026-08-17

1. Решение

	Выбор	Одной строкой почему
Язык	Rust	Один статический бинарник на все платформы и одна реализация протокола и крипто — общее ядро для приложения, iOS-расширения и сервера
База данных	SQLite, вкомпилирована в бинарник (rusqlite bundled)	Масштаб «один пользователь / малый круг»; ноль администрирования; бэкап — один файл
TLS	rustls	Без зависимости от системного OpenSSL — ничего не требуется на машине пользователя
Поставка	Один самодостаточный исполняемый файл на платформу	«Скачал и запустил» — без рантаймов, интерпретаторов и внешних библиотек
Платформы	Windows x64 · macOS arm64/x64 · Linux x64/arm64 · Raspberry Pi (arm64/armv7)	Linux — полностью статические musl-бинарники

Критерии выбора: деплой на все десктопные платформы и ARM; размер дистрибутива; простота установки для нетехнического пользователя; лёгкая и стабильная БД; восстановимость. Владение языком внутри команды из критериев исключено. Сравнение кандидатов — §2.

2. Кандидаты

Легенда:  хорошо ·  с оговорками ·  дисквалифицирует

	Все платформы + Pi	Размер	«Один файл и запустил»	SQLite внутри	Одно ядро с приложением
Rust		 единицы МБ			
Go		 ~12 МБ			
Dart		 ~4-5 МБ			
Python		 десятки МБ			

Последняя колонка — решающая, и она не про сервер. Клиентская часть протокола и криптоядро **обязаны** быть нативными: на iOS содержимое уведомлений забирает Notification Service Extension — отдельный процесс, где Flutter/Dart не работают ([notification-delivery.md](#) §5). Значит нативное ядро существует в любом случае. Написать сервер на том же языке — единственный вариант, при котором протокол реализован **один раз**: ядро подключается к Flutter через FFI, к расширению через Swift-биндинги, к серверу напрямую. Любой другой язык сервера означает вторую параллельную реализацию протокола и её вечную синхронизацию с первой. Это ровно модель libsignal: ядро на Rust, официальные биндинги Swift, Kotlin и TypeScript.

Python отпадает на доставке: нужен либо интерпретатор на машине пользователя, либо PyInstaller-бандл на десятки мегабайт с хрупким запуском. Для нетехнического пользователя оба варианта — не «скачал и запустил».

Dart отпадает на двух пунктах, и владение языком их не компенсирует. Первый — ядро: Dart не запускается в iOS-расширении, значит протокол пришлось бы держать в двух реализациях. Второй — поставка: кросс-компиляция `dart compile exe` работает **только в Linux-цели** (с Dart 3.8/3.9; Windows и macOS собираются только нативно на своих ОС), Linux-бинарник привязан к glibc (musl официально не поддерживается), а однофайловость и автоматический бандл SQLite взаимоисключены — `package:sqllite3` подтягивает нативную библиотеку через hooks, но hooks не работают с `dart compile exe`, а `dart build cli` выдаёт каталог с `.so / .dll` рядом с exe. Серверная экосистема тонкая: shelf зрелый, но минимальный; Serverpod требует PostgreSQL; dart_frog — волонтёрский проект после ухода мейнтейнеров.

Go — сильный второй. Единственное, в чём он лучше Rust: кросс-компиляция всех целей с одной машины штатна (PocketBase собирает все 9 платформенных бинарников на одном Linux-раннере), тогда как Rust собирает macOS-цель только на mac-раннере. ⚠ Для нас эта разница ничего не стоит: mac-раннеры уже есть — приложение собирается на macOS. Против Go: бинарник вдвое-втрое больше, и как язык общего ядра он не подходит — Go-рантайм с GC внутри iOS-расширения с жёстким лимитом памяти никто из мессенджеров не возит, то есть ядро при Go-сервере всё равно было бы на Rust, и протокол снова раздваивается. Go становится правильным выбором, только если требование общего ядра снимется.

3. Почему Rust — подробнее

- **Одна реализация протокола и крипты** на приложение, расширение и сервер (см. §2). Прецедент — libsignal.

- **Поставка.** Полностью статические Linux-бинарники (musl, SQLite и TLS вкомпилированы) — работают на любом дистрибутиве, включая Alpine и старые glibc. Windows-цель собирается даже с Linux-машины; macOS — на mac-раннере.
- **Размер.** Сервер нашего масштаба — единицы мегабайт: сопоставимый по скоупу Rust-сервер (miniserve) — 2–6 МБ несжатого бинарника. Для сравнения: PocketBase (Go) — 11,5–12,5 МБ в архиве.
- **Ресурсы.** Нет GC и рантайма — предсказуемая память на Raspberry Pi-классе железа при 24/7.
- **Безопасность памяти** без рантайма — для постоянно торчащего в сеть сервиса, который обслуживает сам себя без администратора, это класс ошибок, вычеркнутый целиком.
- **ARM первоклассно:** aarch64 — Tier 1 цель Rust; статические arm64-бинарники — обычная практика (семейство Conduit шипит их для Matrix-серверов, работающих на Pi).

⚠ Цена: серверный код на Rust пишется медленнее, чем на Go или Dart, и порог входа выше. Принято осознанно — скорость кодирования исключена из критериев, а сервер маленький (по масштабу ближе к push-шлюзу Snikket, чем к Signal-Server).

4. База данных: SQLite

Масштаб задачи: один пользователь или малый круг, один процесс, единицы одновременных клиентов. Отдельный СУБД-сервер здесь не решает ни одной задачи, но добавляет процесс, который нужно ставить, запускать, обновлять и чинить.

	Выбор / отказ	Почему
SQLite	🟢 выбор	Вкомпилирована в бинарник (<code>rusqlite</code> bundled — статически собирает актуальную SQLite): ни системных зависимостей, ни отдельного процесса. Самая развёрнутая БД в мире, инструменты осмотра везде
PostgreSQL	🔴	Требует установки, службы, обновлений и роли администратора — которой в модели NOX не существует . Оправдан на масштабе компании, не пользователя
Embedded KV (RocksDB, sled, redb)	🔴	Нет SQL и штатных инструментов осмотра; RocksDB заметно раздувает бинарник (Matrix-серверы на нём — 60–105 МБ)
Плоские файлы	🔴	Нет транзакций и запросов; при отключении питания целостность — наша проблема, а не проверенного движка

Режим: WAL + `synchronous=FULL` . Дешёвое железо умеет врать про fsync — гарантии любой БД на нём аннулируются, поэтому источник истины — устройство, а сервер хранит то, что можно вывести заново (границы хранения — Q1).

Восстановимость — свойство выбора, а не отдельная подсистема. Вся база — один файл: бэкап — копия файла (`VACUUM INTO` для консистентного снимка на живом сервере), восстановление — положить файл рядом с бинарником и запустить. Худший случай — потеря узла целиком — по архитектуре развёртывания и так «переразвернуть и спариться заново», а не катастрофа.

5. Прецеденты

Проект	Язык сервера	Хранилище	Масштаб
PocketBase	Go	SQLite, встроена («backend in 1 file»)	self-hosted
ntfy	Go	SQLite	self-hosted
Conduit / Tuvunel (Matrix)	Rust	встроенная RocksDB/SQLite, один бинарник, работает на Pi	self-hosted
Snikket	Lua (поверх Prosody)	плоские файлы (SQLite — опция)	self-hosted
chatmail (Delta Chat)	Postfix/Dovecot + Python-обвязка	почтовые хранилища Dovecot	self-hosted
SimpleX (SMP)	Haskell	память + журнал	self-hosted
libsignal	Rust (ядро, биндинги Swift/Kotlin/TS)	—	ядро клиента
Signal-Server	Java	DynamoDB + Redis + FoundationDB + S3	компания
Synapse (Matrix)	Python/Rust	PostgreSQL обязателен	компания

Закономерность жёсткая: **всё, что хостят у себя частные лица, — компилируемый язык, один бинарник, встроенное хранилище.** Внешние СУБД и JVM появляются только на масштабе компании. NOX-класс задачи — верхняя половина таблицы.

6. Открытые вопросы

	Вопрос	На ком	Реестр
●	Границы хранения: полный архив, спул с TTL или без состояния — на выбор БД не влияет, влияет на схему	владелец	Q1
●	Криптопровайдер rustls (ring проще в кросс-сборке, aws-lc-rs — post-quantum) и примитивы ядра — решается вместе с протоколом	архитектура	—
●	Входит ли запасной Android-телефон в площадки размещения. Rust проходит в обе стороны (компилируется в .so внутри APK) — язык на этом вопросе больше не висит	владелец	Q10